



Communauté Française de Belgique

Institut des Carrières Commerciales

Ville de Bruxelles

Waterside

Quai de Willebroeck, 33

1000 Bruxelles

Documentation technique

Projet Réservations

Projet d'intégration et de développement

Lisa Veaceslav

2025 – 2026

1. Présentation du projet

Projet Réservations est une application web de gestion et de réservation de spectacles. Elle fournit un catalogue public, des parcours authentifiés, un back-office Django, des fonctions de modération, un flux RSS, des échanges CSV et une API REST sécurisée par JWT.

1.1 Profils fonctionnels

Profil	Fonctions principales
Public	Consulter le catalogue, les spectacles et les représentations.
Membre (<code>MEMBER</code>)	Gérer son profil, réserver et publier un avis lorsqu'il est éligible.
Producteur (<code>PRODUCER</code>)	Gérer ses spectacles et représentations, modérer leurs contenus.
Critique (<code>CRITIC</code>)	Soumettre et gérer des critiques de presse.
Affilié Free	Consulter l'API spectacles avec un maximum de 10 résultats par page.
Affilié Starter	Consulter l'API spectacles avec un maximum de 25 résultats par page.
Affilié Premium	Consulter l'API spectacles avec un maximum de 100 résultats par page.
Personnel/administrateur	Administrer le catalogue, les rôles, le CSV et la modération globale.

2. Méthodologie et techniques de développement

Le projet suit l'architecture MVT de Django :

- les modèles décrivent le domaine et les contraintes de données ;
- les vues appliquent les règles métier et orchestrent les traitements ;
- les templates génèrent l'interface HTML ;
- les fichiers `urls.py` assurent le routage ;
- les formulaires Django réalisent la validation serveur ;
- l'ORM Django est utilisé pour les accès PostgreSQL ;
- les migrations versionnent le schéma relationnel ;
- JavaScript et `fetch` assurent les traitements AJAX ;
- Django REST Framework sérialise les ressources JSON de l'API.

2.1 Gestion du projet

J'ai organisé le développement par fonctionnalités : mapping, catalogue, réservations, avis, rôles métier, CSV, RSS, critiques de presse, API et interface graphique. J'ai développé chaque évolution sur une branche Git dédiée. J'ai utilisé des messages de commit courts, avec des préfixes comme `feat`, `fix`, `docs` ou `merge` pour indiquer la nature du changement.

Avant chaque fusion, j'ai vérifié le fonctionnement de la fonctionnalité, les permissions concernées et les principaux risques de régression. J'ai ensuite fusionné les changements validés dans `main`, qui constitue la branche de livraison et de déploiement Render. J'ai utilisé GitHub pour centraliser le code et conserver l'historique des modifications. J'ai effectué le suivi par lots fonctionnels, avec une vérification manuelle et des tests Django pour les règles métier sensibles.

3. Environnement technique

Composant	Version ou utilisation
Python	3.12
Django	5.2.8
Django REST Framework	3.16.1
SimpleJWT	5.5.1
PostgreSQL	Base relationnelle locale et base managée Render
psycopg2-binary	2.9.11
python-dotenv	1.1.1
dj-database-url	3.1.2
Gunicorn	25.1.0, serveur WSGI de production
WhiteNoise	6.12.0, fichiers statiques en production
Bootstrap	5, interface responsive
Git/GitHub	Versioning et dépôt distant
Render	Hébergement du service web et de PostgreSQL

Pour le développement, j'ai principalement utilisé PyCharm, le terminal macOS, Git, GitHub et PostgreSQL 16. J'ai contrôlé l'interface et les requêtes réseau avec Google Chrome et Firefox, notamment à l'aide de leurs outils de développement. J'ai utilisé Microsoft Word pour la mise en page et l'export de cette documentation.

4. Architecture du dépôt

```
reservations/
├─ accounts/                # inscription, profil, rôles et validations
├─ api/catalogue/           # vues DRF, permissions, pagination, OpenAPI
├─ catalogue/
│   └─ forms/               # formulaires et validations serveur
│   └─ management/commands/ # données et comptes de démonstration
│   └─ migrations/         # évolution du schéma PostgreSQL
│   └─ models/              # modèles du domaine
│   └─ static/              # CSS, JavaScript et images
│   └─ templates/           # templates HTML
│   └─ views/               # vues organisées par ressource
├─ reservations/           # réglages, routes racines, WSGI et ASGI
├─ build.sh                 # construction du service Render
├─ render.yaml              # Blueprint Render
├─ manage.py
├─ requirements.txt
└─ .env.example
```

5. Modèle de données

Les principales entités sont :

- `User` et `UserMeta` : identité Django et préférences de profil ;

- `Artist`, `Type`, `ArtistType` et `ArtistTypeShow` : artistes et fonctions ;
- `Locality` et `Location` : localités et lieux ;
- `Show`, `Price` et `Representation` : catalogue, tarifs et séances ;
- `Reservation` et `RepresentationReservation` : réservation, quantité et prix

figé au moment de l'opération ;

- `Review` : note et commentaire d'un membre ;
- `PressReview` : critique de presse liée à un spectacle.

Relations métier structurantes :

- un spectacle possède plusieurs représentations et plusieurs tarifs ;
- une réservation appartient à un utilisateur et porte sur une ou plusieurs

représentations via un modèle intermédiaire ;

- un spectacle peut avoir plusieurs producteurs ;
- un utilisateur ne peut publier qu'un avis par spectacle ;
- avis et critiques suivent un état de modération ;
- les contraintes Django utilisent notamment `UniqueConstraint`, `CASCADE`,

`RESTRICT` et `SET_NULL` selon la durée de vie métier des relations.

6. Fonctionnalités implémentées

6.1 Comptes et authentification

- inscription avec identité, e-mail, login, langue et mot de passe ;
- attribution automatique du rôle membre ;
- politique de mot de passe Django complétée par une majuscule et un caractère

spécial obligatoires ;

- vérification AJAX de la disponibilité du login et de l'e-mail ;
- connexion, déconnexion et gestion du profil ;
- récupération du mot de passe au moyen des vues et jetons Django ;
- e-mails affichés dans le terminal avec le backend console actuel.

6.2 Catalogue

- catalogue paginé par 10 résultats ;
- recherche textuelle ;
- filtre par lieu et disponibilité réelle ;
- tri par titre, lieu, disponibilité et prix ;
- conservation des paramètres pendant la pagination ;
- pages de détail pour les spectacles et représentations ;
- CRUD des artistes, types, localités, lieux, tarifs, spectacles et

représentations selon les permissions.

Un spectacle est réellement reservable uniquement si l'option administrative `bookable` est active, si un tarif existe et si une représentation future est disponible.

6.3 Réservations, avis et modération

- réservation d'une représentation future avec choix du tarif et d'une quantité

comprise entre 1 et 20 ;

- historique personnel des réservations ;
- avis autorisé seulement après une représentation passée réellement réservée

et non annulée ;

- note de 1 à 5 et commentaire ;
- remise en attente après création ou modification ;
- modération AJAX des avis par le personnel ou le producteur concerné ;
- création de critiques de presse par les critiques ;
- modération des critiques par le personnel ou le producteur concerné.

6.4 Administration et échanges

- interface Django Admin sécurisée ;
- gestion dynamique du catalogue ;
- attribution des rôles métier par le superutilisateur ;
- import et export du catalogue des spectacles au format CSV ;
- flux RSS des vingt prochaines représentations ;
- API REST artistes et spectacles ;
- documentation OpenAPI JSON et interface Swagger.

7. URLs fonctionnelles principales

La racine de production est <https://reservations-django.onrender.com>.

URL	Fonction	Accès principal
/	Accueil	Public
/accounts/signup/	Inscription	Public
/accounts/signup/availability/	Validation AJAX login/e-mail	Public
/accounts/login/	Connexion	Public
/accounts/logout/	Déconnexion	Authentifié
/accounts/password_reset/	Demande de réinitialisation	Public
/accounts/profile/	Profil	Authentifié
/accounts/roles/	Attribution des rôles	Superutilisateur
/catalogue/show/	Catalogue des spectacles	Public
/catalogue/show/<id>	Détail d'un spectacle	Public
/catalogue/representation/	Liste des représentations	Public
/catalogue/representation/<id>	Détail d'une représentation	Public
/catalogue/representation/<id>/reserve	Réserver	Authentifié
/catalogue/reservation/	Mes réservations	Authentifié
/catalogue/show/<id>/review	Publier un avis	Membre éligible
/catalogue/reviews/moderation/	Modération des avis	Staff/producteur
/catalogue/press-reviews/	Critiques de presse	Selon rôle
/catalogue/press-reviews/moderation/	Modération presse	Staff/producteur
/catalogue/csv/shows/import/	Import CSV	Personnel
/catalogue/csv/shows/export/	Export CSV	Personnel
/catalogue/rss/representations/	Flux RSS	Public
/catalogue/api/docs/	Documentation Swagger	Public
/catalogue/api/schema/	Schéma OpenAPI JSON	Public
/admin/	Back-office Django	Staff
/admin/password_reset/	Mot de passe administrateur oublié	Public

Les ressources `artist`, `type`, `locality`, `location`, `price`, `show` et `representation` possèdent également des routes de liste, détail, création, modification et suppression sous `/catalogue/`.

8. Authentification, autorisations et sécurité

8.1 Mécanismes appliqués

- mots de passe hashés par le système d'authentification Django ;
- sessions Django pour le front-office et l'administration ;
- décorateurs et règles métier pour les permissions ;
- restrictions par groupes `MEMBER`, `PRODUCER`, `CRITIC` et affiliés ;
- middleware CSRF actif et envoi de `X-CSRFToken` par les appels AJAX ;

- échappement automatique des templates contre les injections XSS usuelles ;
- ORM Django et requêtes paramétrées contre les injections SQL ;
- cookies `Secure` et `SameSite` configurables ;
- HTTPS forcé en production Render ;
- prise en charge du proxy HTTPS de Render ;
- `ALLOWED_HOSTS` et `CSRF_TRUSTED_ORIGINS` complétés avec le nom public Render ;
- clé Django et identifiants PostgreSQL fournis par variables d'environnement ;
- fichier `.env` local ignoré et exemple sans secret dans `.env.example`.

HSTS est disponible mais doit être activé uniquement après validation durable du domaine et de tous ses sous-domaines.

8.2 Sécurité de l'API

L'API accepte Session Authentication, Basic Authentication et JWT. Avec JWT, le client transmet `Authorization: Bearer <access_token>`. Les requêtes fondées sur la session conservent la protection CSRF. Les écritures sur les spectacles sont réservées au personnel et leur lecture aux affiliés autorisés.

9. Procédures d'identification

9.1 Utilisateur public et membre

Un visiteur crée son compte sur `/accounts/signup/`, puis se connecte sur `/accounts/login/`. Le groupe `MEMBER` est attribué à l'inscription.

9.2 Personnel et administrateur

Un superutilisateur est créé avec :

```
python manage.py createsuperuser
```

Il peut ensuite se connecter sur `/admin/`. En production, le mot de passe doit être fort, unique et transmis séparément de cette documentation.

9.3 Comptes locaux de démonstration

Ces comptes sont exclusivement réservés au développement local :

Profil	Login	Mot de passe temporaire
Membre	<code>demo_member</code>	<code>DemoMember!2026</code>
Producteur	<code>demo_producer</code>	<code>DemoProducer!2026</code>
Critique	<code>demo_critic</code>	<code>DemoCritic!2026</code>
Affilié Free	<code>demo_affiliate_free</code>	<code>DemoFree!2026</code>
Affilié Starter	<code>demo_affiliate_starter</code>	<code>DemoStarter!2026</code>
Affilié Premium	<code>demo_affiliate_premium</code>	<code>DemoPremium!2026</code>
Administrateur	<code>demo_admin</code>	Généré et transmis séparément

La commande `python manage.py create_demo_accounts` crée ou actualise ces comptes. Elle refuse de s'exécuter lorsque `DEBUG=False`, afin d'empêcher leur installation accidentelle en production.

Lors de la première exécution, le mot de passe temporaire de `demo_admin` est affiché dans le terminal. Je le conserve pour l'évaluation et je le transmets séparément de la documentation. Si nécessaire, je peux créer un nouvel

administrateur avec `python manage.py createsuperuser`.

10. API consommée

Aucun service web tiers n'est actuellement consommé par l'application. Cette exigence reste un développement futur. L'API REST décrite ci-dessous est produite par le projet et ne doit pas être présentée comme une API externe consommée.

11. API REST produite

11.1 Authentification

Méthode	Endpoint	Fonction
POST	<code>/catalogue/api/token/</code>	Obtenir les jetons <code>access</code> et <code>refresh</code>
POST	<code>/catalogue/api/token/refresh/</code>	Renouveler le jeton d'accès

Exemple :

```
curl -X POST https://reservations-django.onrender.com/catalogue/api/token/ \
-H "Content-Type: application/json" \
-d '{"username":"mon-utilisateur","password":"mon-mot-de-passe"}'
```

11.2 Ressources

Méthodes	Endpoint	Autorisation
GET, POST	<code>/catalogue/api/artists/</code>	Permissions Django du modèle
GET, PUT, PATCH, DELETE	<code>/catalogue/api/artists/<id>/</code>	Permissions Django du modèle
GET, POST	<code>/catalogue/api/shows/</code>	Affilié en lecture, staff en écriture
GET, PUT, PATCH, DELETE	<code>/catalogue/api/shows/<id>/</code>	Affilié en lecture, staff en écriture

L'endpoint spectacles accepte :

- `q` pour la recherche ;
- `location` pour le slug du lieu ;
- `reservable=true|false` pour la disponibilité réelle ;
- `ordering` avec `title`, `created_in`, `duration`, `price` ou `availability` ;
- `page` et `page_size` pour la pagination limitée par niveau affilié.

Les réponses incluent des liens HATEOAS vers la ressource, la collection et la page HTML du spectacle. La documentation interactive est disponible sur `/catalogue/api/docs/` et le schéma sur `/catalogue/api/schema/`.

12. Flux RSS

`/catalogue/rss/representations/` produit un flux RSS 2.0 contenant au maximum vingt représentations futures, triées chronologiquement. Chaque entrée fournit le spectacle, la date, le lieu, une description et un lien vers le détail.

13. Import et export CSV

Les opérations sont réservées au personnel :

- l'export écrit un CSV UTF-8 compatible avec Excel ;
- l'import accepte un fichier `.csv` de 2 Mo maximum ;
- toutes les lignes sont validées avant l'écriture ;
- l'import est exécuté dans une transaction atomique ;
- les colonnes sont `slug`, `title`, `description`, `duration`, `created_in`,

`location_slug` et `bookable`.

14. Installation locale

14.1 Prérequis

- Python 3.12 ;
- PostgreSQL ;
- Git.

14.2 Installation à partir de l'archive

Après avoir téléchargé l'archive remise avec le projet, je l'extrais puis j'ouvre un terminal dans le dossier obtenu. Le nom exact de l'archive peut varier :

```
unzip reservationsDjango.zip
cd reservationsDjango
python3 -m venv .env
source .env/bin/activate
python -m pip install -r requirements.txt
cp .env.example .env
```

Je crée ensuite une base et un utilisateur PostgreSQL. Les valeurs peuvent être adaptées, mais elles doivent correspondre à celles du fichier `.env` :

```
CREATE USER reservations_user WITH PASSWORD 'mot-de-passe-local';
CREATE DATABASE reservations OWNER reservations_user;
```

Dans `.env`, je renseigne au minimum :

```
DJANGO_SECRET_KEY=une-cle-locale-longue-et-aleatoire
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
POSTGRES_DB=reservations
POSTGRES_USER=reservations_user
POSTGRES_PASSWORD=mot-de-passe-local
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
```

J'initialise enfin l'application et je démarre le serveur :

```
python manage.py migrate
python manage.py runserver
```

Le site est alors disponible à l'adresse <http://127.0.0.1:8000/>. Le fichier `.env` contient des secrets locaux et ne doit jamais être ajouté au dépôt.

Pour charger les données de démonstration en environnement local :

```
python manage.py seed_demo_catalogue --username demo_member
python manage.py create_demo_accounts
```

15. Déploiement Render

15.1 URLs de livraison

- **Site en ligne** : <https://reservations-django.onrender.com>
- **Dépôt GitHub** : <https://github.com/slavaBJJ/reservationsDjango>
- **Branche de livraison** : `main`
- **Swagger** : <https://reservations-django.onrender.com/catalogue/api/docs/>
- **OpenAPI** : <https://reservations-django.onrender.com/catalogue/api/schema/>
- **RSS** : <https://reservations-django.onrender.com/catalogue/rss/representations/>

15.2 Procédure

Le fichier `render.yaml` décrit un service web `reservations-django` et une base PostgreSQL `reservations-db`. Render génère `DJANGO_SECRET_KEY`, injecte `DATABASE_URL` et fournit `RENDER_EXTERNAL_HOSTNAME`.

À chaque déploiement, `build.sh` exécute :

1. `pip install -r requirements.txt ;`
2. `python manage.py collectstatic --no-input ;`
3. `python manage.py migrate.`

Le service démarre avec :

```
gunicorn reservations.wsgi:application
```

WhiteNoise sert les fichiers statiques collectés. Le système de fichiers du service est éphémère : les données persistantes doivent rester dans PostgreSQL. Sur l'offre gratuite, le service peut se mettre en veille et la base PostgreSQL expire après trente jours ; une offre payante est nécessaire pour une conservation durable.

16. Configuration par variables d'environnement

Variable	Utilisation
DJANGO_SECRET_KEY	Clé cryptographique, obligatoire et secrète
DJANGO_DEBUG	True en local, False en production
DJANGO_ALLOWED_HOSTS	Hôtes autorisés hors ajout automatique Render
DJANGO_CSRF_TRUSTED_ORIGINS	Origines HTTPS autorisées
DJANGO_LANGUAGE_CODE	Langue de l'application
DJANGO_TIME_ZONE	Fuseau horaire
DATABASE_URL	Connexion PostgreSQL Render
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Connexion locale
POSTGRES_HOST, POSTGRES_PORT	Serveur PostgreSQL local
DJANGO_CSRF_COOKIE_SECURE	Cookie CSRF limité à HTTPS
DJANGO_SESSION_COOKIE_SECURE	Cookie de session limité à HTTPS
DJANGO_SECURE_SSL_REDIRECT	Redirection HTTP vers HTTPS
DJANGO_SECURE_HSTS_SECONDS	Durée HSTS après validation du domaine

17. Contrôles techniques

Commandes utiles :

```
python manage.py check
python manage.py showmigrations
python manage.py collectstatic --no-input
git diff --check
```

La recette manuelle doit couvrir au minimum l'inscription, la connexion, le mot de passe oublié, la recherche catalogue, la réservation, l'avis, la modération, les rôles, le CSV, le RSS, l'administration et l'API JWT.

18. Limites et développements futurs

- intégrer une API culturelle tierce et documenter son URL ;
- ajouter des statistiques de ventes pour les producteurs ;
- configurer un serveur SMTP pour les e-mails de production ;
- ajouter un paiement en ligne si le périmètre est étendu ;
- mettre en place sauvegardes, supervision et rotation des secrets ;
- augmenter la couverture des tests et automatiser leur exécution avec une

Conclusion

Le projet couvre le catalogue, les comptes, les réservations, les avis, les rôles métier, la modération, le CSV, le RSS, l'administration et une API REST authentifiée. Il est déployé sur Render depuis la branche `main`. Les principales extensions encore identifiées sont la consommation d'une API tierce, les statistiques producteur et la configuration d'un service d'e-mail de production.